

Prototype Implementation of IC3

Lucky M. Kispotta

Advisor: Prof. MK Srivas

Abstract

We present a clean and robust implementation of Property Directed Reachability (PDR) Algorithm for verifying safety properties of finite-state boolean systems. Our implementation is based on the original paper by Aaron Bradely. We have implemented the algorithm and some optimizations in a tool called *ICEpie*[8]. The tool is written in Python and uses *z3* as the backend solver. The implementation is inspired by the paper Efficient Implementation of Property Directed Reachability [2].

1 Introduction

Property Directed Reachability (PDR) or IC3 (**I**ncremental **C**onstruction **I**nductive **C**lauses **I**ndubitable **C**orrectness). [1]

1. Incremental Construction - Frames are created incrementally and a frame k is a one-step over approximation of the states reachable in $\leq k$ transitions from the initial state.
2. Inductive Clauses - Propagate inductive clauses from a "smaller frame" to a "bigger" frame.
3. Indubitable Correctness - The algorithm is sound and complete.

IC3 is one of the most scalable and efficient algorithm for proving safety of a transition system. In this report we elaborate the implementation details and it's technical and theoretical nuances.

2 Preliminaries

Definition 1 (Finite State Machine). A *finite state machine* (FSM) is a tuple (S, I, T, P) where S is a finite set of states, $I(s)$ is the initial-state predicate or set of states, $T(s, s')$ is the transition relation, and $P(s)$ is the safety property. We wish to verify that no state reachable from I violates P .

Definition 2 (Cube and Clause). A *literal* is a boolean variable or its negation. A *cube* is a conjunction of literals. A state is usually represented as a *cube*. A *clause* is the negation of a cube (a disjunction of negated literals). A *minterm* is a cube containing all state variables.

Definition 3 (Frames). IC3 maintains a sequence of frames $[F_0, F_1, \dots, F_N]$ with the following invariant properties:

1. $F_0 = I$.
2. Each frame F_k ($k \geq 1$) is a conjunction of clauses.
3. $F_k \Rightarrow F_{k+1}$. The frames preserve monotonicity
4. $F_k \wedge T \Rightarrow F'_{k+1}$.

5. $F_k \Rightarrow P$ for $k < N$.

Definition 4 (Proof Obligation). A *proof obligation* is a tuple (s, k) containing the frame number k and a cube s . Here s is the bad state which needs to be blocked at frame k .

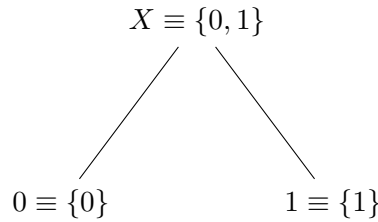
Definition 5 (Ternary Simulation [7]). Three valued $\{0, 1, X\}$ logical circuits can be used to model the transition system. The inputs can then be assigned *unknown* value (X). This optimization allows computations at a faster speed eliminating the need for *SAT* calls. The tables below depict the behaviour of *AND*, *OR* and *NOT* gates on the ternary domain.

a	$\neg a$
0	1
1	0
X	X

a	b	$a \wedge b$
0	0	0
0	1	0
0	X	0
1	0	0
1	1	1
1	X	X
X	0	0
X	1	X
X	X	X

a	b	$a \vee b$
0	0	0
0	1	1
0	X	X
1	0	1
1	1	1
1	X	1
X	0	X
X	1	1
X	X	X

The idea is that X is less precise than 0 and 1. This naturally forms an ordering. The lattice below shows the abstract domain used for ternary simulation.



The join operation corresponds to merging information from multiple execution paths. For example,

$$0 \sqcup 1 = X$$

since

$$0 \cup 1 = 0, 1$$

Thus, ternary simulation can be viewed as a lightweight form of abstract interpretation, where multiple Boolean states are compactly represented and propagated simultaneously through the circuit.

3 Algorithm Overview

3.1 Main Loop

The main PDR loop (Algorithm 1) repeatedly:

1. Creates a new frame F_N .
2. Finds bad states in F_N (states violating P) and blocks them via recursive (or iteratively) proof obligations.
3. Generalizes blocked cubes.
4. Propagates blocked cubes forward.
5. Checks for a fixpoint (two identical adjacent frames).

Algorithm 1 Main PDR procedure

Require: Transition T , initial states I , property P , variables S

Ensure: Invariant or counterexample

```

1: Check  $I \Rightarrow P$ ; if not, return CEX
2: loop
3:   Extend to a new frame  $F_N$ 
4:   while  $\text{SAT}(F_N \wedge \neg P)$  do
5:     Extract bad cube  $s$  from model
6:     if  $\neg \text{BLOCKCUBE}(s, N)$  then
7:       return CEX
8:     end if
9:   end while
10:   $\text{GENERALIZE}(\text{using } \textit{unsat-core})$ 
11:   $\text{PROPAGATE}$ 
12:  if  $\exists k : F_k = F_{k-1}$  then
13:    return invariant  $F_k$ 
14:  end if
15: end loop

```

Note: The termination condition is when the algorithm finds a fixpoint. Since the implementation stores the δ or the difference between the two frames (*delta encoded frames*), the termination condition used in the program is $F_k = \phi$.

3.2 Blocking Phase

Given a proof obligation (s, k) , a cube s that must be blocked at frame k . The algorithm checks the SAT query:

$$\text{SAT?}(F_{k-1} \wedge \neg s \wedge T \wedge s') \quad (\text{Q2})$$

If UNSAT, then s is blocked: $\neg s$ is added as a clause to R_k (and all preceding frames, by containment). Before adding, the cube is *generalized* by removing unnecessary literals. The generalization at this phase is done using *unsat-core*.

If SAT, a predecessor m is extracted and a new proof obligation $(m, k-1)$ is created. If a proof obligation reaches frame F_0 , it is a real counterexample.

This phase is executed iteratively in the implementation. The reasons iterative loop is preferred over a recursive algorithm are :

1. Memory efficient - The recursive approach generates call stacks for each call while the iterative method uses only one.
2. Time efficient - Calling functions is generally expensive as it requires pushing stack frames, passing parameters and returning value.

Algorithm 2 BLOCKCUBE

Require: Bad cube s_0 , frame k_0

```

1:  $Q \leftarrow$  priority queue ordered by frame (min first)
2:  $Q.\text{push}(s_0, k_0)$ 
3: while  $Q \neq \emptyset$  do
4:    $(s, k) \leftarrow Q.\text{popMin}()$ 
5:   if  $k = 0$  then return false ▷ CEX
6:   end if
7:   if  $s$  already blocked at frame  $k$  then continue
8:   end if
9:    $\text{result} \leftarrow \text{SOLVERELATIVE}(s, k)$ 
10:  if  $\text{result}$  is UNSAT then
11:     $s_g \leftarrow \text{GENERALIZE}(s, k)$ 
12:    for  $i \in [1, k]$  do block  $s_g$  in frame  $i$ 
13:    end for
14:    if  $k < N$  then  $Q.\text{push}(s_g, k+1)$ 
15:    end if
16:  else
17:    Extract predecessor  $m$  from SAT model
18:     $Q.\text{push}(m, k-1)$ ;  $Q.\text{push}(s, k)$ 
19:  end if
20: end while
21: return true

```

3.3 Generalization [3]

The algorithm returns a counterexample at a frame. The given CTI or blocked cube can be generalized to include a bigger set of state, resulting in faster execution. The counterexample is a cube of literals.

Generalization is done at two places :

1. Generalizing the CTI - This blocks all cubes or states that are produced when the SAT at :

$$\{s \mid \text{SAT?}(F_{k-1} \wedge \neg s \wedge T \wedge s')\} \quad (1)$$

A *naive* way to do this is to remove literals one-by-one from the blocked clause and check if it is still SAT.

Algorithm 3 GENERALIZE - NAIVE

Require: Cube s , frame k

```

1: for each literal  $\ell \in s$  do
2:   if  $|s| \leq 1$  then break
3:   end if
4:    $s' \leftarrow s \setminus \{\ell\}$ 
5:   if  $\neg \text{ISINITIAL}(s')$  and  $\text{SOLVERELATIVE}(s', k) = \text{UNSAT}$  then
6:      $s \leftarrow s'$ 
7:   end if
8: end for
9: return  $s$ 

```

Given a cube s' be a *CTI* at frame k . Ternary Simulation provides a very efficient way to drop literals in a cube without any *SAT* calls. The idea is to replace a literal in the predecessor with X and check by simulating the circuit whether the next state in the transition is s' .

Suppose the SAT solver returns a predecessor state

$$s : (l_1, l_2, l_3) = (0, 1, 0)$$

for the CTI

$$c : (l'_1 = 1).$$

Since the CTI constrains only the next-state variable l'_1 , only the corresponding transition function

$$T(l_1) = l_2 \vee l_3$$

needs to be analyzed.

We now attempt to generalize the predecessor cube by replacing literals with X .

Consider replacing l_1 with X :

$$(X, 1, 0).$$

Evaluating the transition function under ternary semantics gives

$$1 \vee 0 = 1.$$

Thus, regardless of the value of l_1 , the next-state value l'_1 remains forced to 1. Therefore, the literal corresponding to l_1 is irrelevant and can be removed.

The predecessor cube is generalized to

$$(l_2, l_3) = (1, 0).$$

Algorithm 4 GENERALIZE-TERNARY

Require: Predecessor cube s , CTI s'

```

1: for each literal  $\ell \in s$  do
2:   if  $|s| \leq 1$  then
3:     break
4:   end if
5:    $gen_s \leftarrow s$ 
6:   Replace literal  $\ell$  in  $gen_s$  with  $X$ 
7:   if TERNARYSIM( $gen_s, s'$ ) then
8:      $s \leftarrow gen_s$ 
9:   end if
10: end for
11: return  $s$ 
```

Here, TERNARYSIM(s, s') evaluates the transition relation under ternary semantics. The procedure returns true if the generalized predecessor cube gen_s still forces the CTI s' in the next state.

2. Generalizaing the blocked clause - The intuition here is given a blocked cube s' , can we block multiple states at once by dropping literals ? This is done using *unsat-core*.

Given a cube $s' = l_1 \wedge l_2$ add activation literals to them which will be set to true when they are part of the *unsat-core*.

$$act_1 \iff l_1 \quad act_2 \iff l_2$$

Remove literals whose corresponding activation literals are false in *unsat-core*. Blocking the resulting literal will imply blocking all unreachable states.

Algorithm 5 GENERALIZE-UNSATCORE

Require: Blocking cube s , frame k

Ensure: Generalized blocking cube

- 1: Introduce activation literals act_ℓ for each literal $\ell \in s$
- 2: **for** each literal $\ell \in s$ **do**
- 3: Add assumption constraint

$$act_\ell \Rightarrow \ell$$

4: **end for**

5: Query SAT solver:

$$F_k \wedge T \wedge s \wedge \neg s'$$

under assumptions

$$act_\ell \mid \ell \in s$$

6: **if** query is UNSAT **then**

7: $U \leftarrow$ UNSAT core returned by solver

8: **end if**

9: $g \leftarrow \emptyset$

10: **for** each literal $\ell \in s$ **do**

11: **if** $act_\ell \in U$ **then**

12: $g \leftarrow g \cup \ell$

13: **end if**

14: **end for**

15: **return** g

3.4 Forward Propagation

After each major iteration, blocked cubes are pushed forward (Algorithm 6). For each cube c in F_k , we check whether $F_k \wedge T \wedge c'$ is UNSAT (using query Q1, without the inductive $\neg c$ term, since c is already present in frame k). If so, c is moved to frame $k+1$. If F_k becomes empty, then $F_k = F_{k+1}$ and we have a fixpoint.

Algorithm 6 PROPAGATE

1: **for** $k = 1$ **to** $N-1$ **do**

2: **for** each cube $c \in F_k$ **do**

3: **if** UNSAT($R_k \wedge T \wedge c'$) **then**

4: Move c from F_k to F_{k+1}

5: **end if**

6: **end for**

7: **if** $F_k = \emptyset$ **then return true**

▷ Fixpoint

8: **end if**

9: **end for**

10: **return false**

To summarize the idea, the inductive clause are propagated to later frames. This allows faster convergence.

4 Implementation Details

4.1 Design Decisions

Our implementation makes three key architectural choices, summarized in Table 1.

Table 1: Summary of implementation.

Aspect	Details
Solver instances	Single solver, incremental (activation literals)
Frame storage	Delta-encoded, R_k reconstructed on demand
Blocking	Priority queue (BLOCKCUBE)
Variables	Z3 Bool, separate present/next copies
Cube representation	Python <code>dict{str: bool}</code>
Generalization	Ternary simulation + literal removal optional unsat-core
Subsumption	In <code>add_blocked_cube</code>
Containment	Implicit: F_k active in all $R_{j \leq k}$

4.2 Single Incremental Solver

Only one instance of solver is maintained in the entire lifetime of the algorithm. The switching on or off of clauses in the solver are done through *activation* literals. This allows internal learned clauses to be preserved. The solver gets progressively "*smarter*" about the transition relation and the frames. For example,

The transition system is added to the solver and is gated by an *activation* literal.

```

1 self.act_T = Bool('__trans_act')
2 self.solver.add(Implies(self.act_T, ckt.T()(self.vd, self.vdp)))

```

So when required the transition system T can be activated by setting act_T to true. Every query includes T which is gated.

$$act_T \implies T$$

Also, all frames are added to the solver and similarly multiple activation literals are used to *switch on/off* the required frames. Each clause is stored exactly once. Since frames are conjunction of *not* of the cubes.

$$act_k \implies \neg c$$

The above formula allows activation of *not* of cube c when frame k is active. Assuming $\neg c$ is contained inside frame k .

4.3 Delta-Encoded Frames

Instead of storing the full clause set for a frame in a list, store each clause once, at the highest level it was learned.

We maintain a set of delta frames R_k

We store blocked cubes in a *delta-encoded* trace. Each R_k stores only the cubes blocked at frame k , not duplicated in earlier frames. The over-approximation is reconstructed on demand:

$$F_k = \bigwedge_{i \geq k} \neg R_i$$

```

1 def get_R(self, k, vd):
2     if k == 0:
3         return self.init_expr
4     clauses = []
5     for i in range(k, len(self.F)):
6         for cube in self.F[i]:
7             clauses.append(negate_cube(cube, vd))
8     return And(*clauses) if clauses else BoolVal(True)

```

The solver doesn't use this function for *SAT* queries. It only requires the entire frame as a formula when outputting the *invariant* (which is a frame) at fixpoint. Instead the solver adds the cube once guarded by the activation literal of that frame (as discussed in the previous section.)

```

1 def _assume_R(self, k):
2     return [act for i, act in self.acts.items() if i >= k]

```

The above function returns the list of activation literals from $k \rightarrow N$. The program adds these in the query to make that frame active.

4.4 AIGER

The *AIGER* format is a compact hardware verification format used widely in model-checking and *SAT* based frameworks like *IC3*. It represent digital circuits as *AND-Inverter graphs* where the system is modeled only using *AND* and *NOT* gates. An *AIGER* model typically consists of inputs, latches (state variables), outputs, and *AND* gates, allowing both combinational and sequential circuits to be represented uniformly.

The tool implemented takes input in *AIGER* format. The format is popular hence a lot of benchmarks were accessible. The format represents a boolean circuit using inputs and latches hence it can be efficiently translated to *AND* formulas.

A sequential circuit in *AIGER* naturally corresponds to a transition system. The *inputs* represent external variables, the *latches* represent the current state of the system, and the next-state functions of the latches define the transition relation. For example, a latch storing a Boolean variable x together with a next-state function $x' = x \oplus in$ describes how the system evolves from one state to another based on the input in . Thus, each clock cycle corresponds to a transition between states in the underlying transition system.

1. **Inputs** - The tool takes a *transition system*, a *property* P and a list of *environment invariant* as the input.
2. **Outputs** - The tool visually outputs whether the property was *safe* or *unsafe*. It also outputs a log containing detailed information about the number of frames used, the contents of each frame, the invariant found etc.

4.5 Environment Invariant

Environment invariants are assumptions regarding the environment given by the user as the input. This is a part of *AIGER* specification. It dictates how the environment should behave. Without these the model could allow unreasonable values in inputs leading to a spurious counterexample. The algorithm will still acknowledge as real.

Given a formula env representing the environment constraint. One way to attach it to the transition system is directly adding it to T .

$$T' = T \wedge env$$

This is undesirable as it requires modifying the transition system. The tool uses latches to monitor this. A `__viol__` literal is added to each state. Given a state, the corresponding `__viol__` literal tracks whether there is a path from the initial state to this state such that there exist a state which has violated the environment invariant. `__viol__` acts like a sticky latch. Every transition includes:

$$viol' = viol \vee \neg env$$

4.6 Cube Representation

We represent cubes as plain Python dictionaries `{var_name: bool}`:

```
1 # Cube: {'b0': True, 'b1': False} means b0=1, b1=0
```

Z3 expressions are built *on demand* by passing variable dictionaries:

```
1 def cube_to_expr(cube, vd):
2     lits = []
3     for v, val in cube.items():
4         lits.append(vd[v] if val else Not(vd[v]))
5     return And(*lits) if lits else BoolVal(True)
```

Subsumption checking uses Python's built-in dictionary view comparison:

```
1 def subsumes(a, b):
2     return a.items() <= b.items()
```

This returns `True` when every key-value pair in `a` also appears in `b`, correctly capturing the notion that a shorter cube (fewer literals) subsumes a longer one.

4.7 Boolean Domain Restriction

Our implementation restricts to Z3 `Bool` variables. With n boolean variables, the state space has exactly 2^n states, guaranteeing that the enumeration of bad cubes terminates.

5 Correctness Arguments

5.1 Soundness

When PDR returns an invariant F_k , the trace properties guarantee:

1. $I \Rightarrow F_k$ (from $F_0 = I$ and containment).
2. $F_k \wedge T \Rightarrow F'_{k+1}$ (consecution).
3. $F_k = F_{k+1}$ (fixpoint, since $F_k = \emptyset$).
4. $F_k \Rightarrow P$.

Combining (1)–(4), F_k is an inductive invariant proving P .

When PDR returns a counterexample that is when the proof obligation reaches frame 0. The counterexample is a trace.

5.2 Containment

When a cube s is blocked at frame k (i.e., $F_{k-1} \wedge \neg s \wedge T \Rightarrow \neg s'$ is proved), we add $\neg s$ as a clause to all frames $F_1, \dots, F_k$. This is sound because $F_i \Rightarrow F_{k-1}$ for $i < k-1$ (containment), so the query remains UNSAT when F_{k-1} is replaced by any stronger F_i .

6 Experimental Validation

We evaluate ICEpie on safety-property benchmarks drawn from the `aiger-safety-properties` suite (T. Niessen) and selected HWMCC instances, parsed directly from ASCII (`.aag`) and binary (`.aig`) AIGER. Each benchmark is run with a timeout. We report three studies: a comparison of PDR against k-induction, an ablation of the two generalization strategies, and an unsat-core sweep on satisfiable (UNSAFE) instances.

6.1 PDR versus k-induction

Table 2 compares convergence depth and runtime for benchmarks on which *both* engines terminate with a definite verdict (60s timeout, $\text{max-}k = 100$). For PDR we report the number of frames built and the fixpoint (inductive-invariant) level; for k-induction we report the converging depth k .

Table 2: PDR vs. k-induction. *fixpoint* = fixpoint level (SAFE only).

Benchmark	Result	L	fixpoint	PDR (s)	k-ind k	k-ind (s)
x-lte-x-squared-32	SAFE	32	1	0.06	0	0.10
traffic-light-prescale-8	SAFE	11	2	0.01	0	0.01
petersons-2t-1core	SAFE	9	7	0.49	37	9.48
petersons-2t-2cores	SAFE	9	6	0.36	43	12.60
tic-tac-toe-3x3	SAFE	21	4	1.84	1	0.04
prime-20996011-acc.	SAFE	25	2	41.49	0	2.57
unary-counter-maximum-8	UNSAFE	8	-	0.01	7	0.02

The two methods are complementary. PDR is dramatically faster on problems with a deep recurrence but a compact inductive invariant: Peterson’s mutual-exclusion protocol converges in 8 frames in under 0.5s, whereas k-induction needs $k = 37\text{--}43$ and 9–13s. Conversely, on shallow properties with an expensive transition relation (`tic-tac-toe`, `prime-accumulator`) k-induction’s single unrolling wins outright, since PDR pays for repeated SAT queries against a large circuit. On counterexamples PDR returns the bad trace without unrolling to the full depth.

6.2 Effect of generalization

We isolate the two generalization techniques: ternary-simulation and unsat-core. By running each benchmark under four configurations (`baseline`, `ternary`, `unsatcore`, `both`) with a 40s timeout. Table 3 reports learned cubes ($\approx$ invariant size), SAT calls, and time for representative SAFE instances.

Three findings stand out. First, the headline benefit is *scalability*: `semaphore-2t-limit-1` (14 latches) does not terminate in the baseline within 40s yet finishes in 7–23s once generalization is enabled. Second, generalization sharply shrinks the inductive invariant: learned cubes drop from $460 \rightarrow 37$ (Peterson), $224 \rightarrow 48$ (Hamming), and $1024 \rightarrow 3$ (traffic-light). Third, the two techniques have different cost profiles: unsat-core consistently reduces SAT calls (it generalizes a cube in one core extraction), while ternary simulation *raises* SAT calls because it probes each latch individually. Generalization is not universally beneficial: on the `fibonacci` family the baseline’s full-minterm clauses converge faster, and the optimized configurations regress or time out, confirming that clause shrinking interacts with invariant shape rather than being a free win.

Table 3: Generalization ablation on SAFE benchmarks. *lrn* = learned cubes, *sat* = SAT calls.

Benchmark (L)	Config	lrn	sat	time (s)
traffic-light-prescale-8 (11)	baseline	1024	2563	1.39
	ternary	3	23	0.01
	unsatcore	82	328	0.09
petersons-2t-2cores (9)	baseline	460	931	0.50
	ternary	41	751	0.53
	unsatcore	45	208	0.06
hamming-distance-07-04 (7)	baseline	224	563	0.26
	ternary	48	772	1.02
	unsatcore	48	171	0.07
semaphore-2t-limit-1 (14)	baseline	timeout > 40 s		
	ternary	799	21112	23.15
	unsatcore	2620	9930	7.20

6.3 Unsat-core on counterexample search

Finally, we sweep unsat-core on/off over tractable UNSAFE HWMCC *microban* [5] instances (Table 4), where the engine must discover a counterexample rather than an invariant.

Table 4: Unsat-core on UNSAFE *microban* instances.

Instance	L	frames		SAT calls	
		no-core	core	no-core	core
microban_1	23	21	21	21422	7724
microban_24	29	22	10	12466	2373
microban_33	35	timeout	11	—	9726

Core-based generalization more than halves SAT calls on counterexample search (21422 $\rightarrow$ 7724 on *microban_1*, 12466 $\rightarrow$ 2373 on *microban_24*) and, on *microban_33*, is the difference between timing out and solving in 11 frames. Generalization helps proof search and refutation alike by keeping the frame clause database compact.

7 Conclusion

We presented a clean, implementation of the PDR algorithm for boolean finite-state systems, based on the efficient formulation of Een et al. [2]. The improvements mentioned in the report address correctness and performance issues in the vanilla implementation while faithfully implementing the algorithm described in the paper.

The implementation correctly handles all test cases: finding counterexamples for reachable bad states and discovering inductive invariants for safe systems. The restriction to boolean domains ensures the algorithm matches the theoretical assumptions under which PDR is guaranteed to terminate.

References

- [1] A. R. Bradley. SAT-based model checking without unrolling. In *Proc. VMCAI*, 2011.

- [2] N. Een, A. Mishchenko, and R. Brayton. Efficient implementation of property directed reachability. In *Proc. FMCAD*, 2011.
- [3] A. R. Bradley and Z. Manna. Checking safety by inductive generalization of counterexamples to induction. In *Proc. FMCAD*, 2007.
- [4] K. L. McMillan. Interpolation and SAT-based model checking. In *Proc. CAV*, 2003.
- [5] HWMCC Benchmarks <https://hwmcc.github.io/2025/>
- [6] Aiger Safety Properties Benchmark <https://github.com/tniessen/aiger-safety-properties>
- [7] Nils Froleyks, Emily Yu, and Armin Biere. Ternary Simulation as Abstract Interpretation. In *Proceedings of the 27th Workshop on Methods and Description Languages for Modelling and Verification of Circuits and Systems (MBMV)*, 2024.
- [8] Lucky Mathias Kispotta ICEpie <https://github.com/Y0maann/IC3>